

دانشگاه صنعتی خواجه نصیرالدین طوسی

K. N. Toosi University of Technology

Faculty of Electrical & Computer Engineering

A Comprehensive Sports Ticket Reservation System

Database Course Project

Team Members: Mahan Rafiee , Hoodad Fallah , Armin SehatZadeh

Instructor: Dr. Pishgoo

Teaching Assistant: Eng. Alireza Ghorbani

Semester: Spring 2026

Project: MahTicket

Sports Ticket Search, Reservation, Payment, and Management Platform

معرفی پروژه

MahTicket یک سامانه جامع برای جستجو، مشاهده، رزرو، خرید، مدیریت و کنسلی بلیط مسابقات ورزشی است.

هدف اصلی سیستم فراهم کردن بستری یکپارچه است که تماشاگر بتواند بدون واسطه مسابقات ورزشی را جستجو کرده، بلیط متناسب با نیاز خود را انتخاب کند، برای مدت محدود رزرو ایجاد کند، پرداخت را انجام دهد و پس از خرید نیز عملیات مرتبط با بلیط مانند مشاهده، کنسلی، استرداد وجه، تغییر جایگاه و گزارش مشکل را مدیریت کند.

سیستم علاوه بر پنل تماشاگر، یک پنل عملیاتی برای کاربران دارای نقش support فراهم می‌کند تا بتوانند رزروها، گزارش‌ها، پرداخت‌های مشکوک، درخواست‌های کنسلی، درخواست‌های تغییر جایگاه و گفتگوهای پشتیبانی را مدیریت کنند.

در طراحی MahTicket صرفاً هدف ایجاد یک رابط نمایشی نبوده است؛ سیستم به صورت چندلایه طراحی شده و شامل موارد زیر است:

- پایگاه داده رابطه‌ای نرمال شده
- Constraints و Trigger های کنترل صحت
- منطق تراکنشی رزرو و پرداخت
- API مستقل و RESTful
- احراز هویت OTP و JWT
- Redis
- Elasticsearch

- Worker مستقل
- رابط کاربری Responsive
- Docker
- تست‌های Unit ، Contract ، Smoke و Integration
- مستند OpenAPI و Postman Collection
- Audit Logging

اهداف پروژه

- اهداف اصلی MahTicket عبارت‌اند از:
- اهداف کاربردی :
- کاربر عادی بتواند:
- حساب کاربری ایجاد کند.
- از طریق Password یا OTP وارد شود.
- پروفایل خود را مشاهده و ویرایش کند.
- مسابقات و بلیط‌ها را جستجو کند.
- بر اساس ورزش، تیم، شهر، ورزشگاه، تاریخ، قیمت، رده و ظرفیت فیلتر کند.
- جزئیات کامل بلیط را مشاهده کند.
- بلیط را برای مدت محدود Hold کند.

- رزرو را پرداخت کند.
- تاریخچه رزرو و خرید را مشاهده کند.
- وضعیت کیف پول و تراکنش‌ها را مشاهده کند.
- درخواست کنسلی ایجاد کند.
- میزان جریمه و مبلغ قابل استرداد را مشاهده کند.
- درخواست تغییر صندلی یا جایگاه ایجاد کند.
- مشکل مربوط به بلیط، رزرو یا پرداخت را گزارش کند.
- مستقیماً با پشتیبانی گفتگو کند.

کاربر پشتیبان بتواند:

- وضعیت سیستم را از Dashboard مشاهده کند.
- رزورها را جستجو و بررسی کند.
- رزرو مشکل‌دار را Flag یا تأیید کند.
- رزرو Hold شده را لغو کند.
- اصلاح ایمن جایگاه انجام دهد.
- درخواست‌های کنسلی را تأیید یا رد کند.
- درخواست‌های تغییر جایگاه را بررسی کند.
- گزارش کاربران را پاسخ دهد و ببندد.
- پرداخت‌های مشکوک را مشاهده کند.
- بلیط ایجاد، ویرایش یا غیرفعال کند.

- کاربر را به شکل کنترل شده غیرفعال کند.

- گفتگوهای پشتیبانی را مدیریت کند.

اهداف فنی :

- طراحی دیتابیس در حداقل ۳ NF

- جلوگیری از Overselling

- حفظ Consistency در عملیات مالی

- انجام عملیات حساس در Transaction

- استفاده از Lock در عملیات رقابتی

- جداسازی Source of Truth از Cache و Search Index

- جلوگیری از استفاده ORM

- استفاده از SQL پارامتری

- کاهش بار PostgreSQL با Redis و Elasticsearch

- ایجاد API های Single Responsibility

- رعایت RBAC

- فراهم کردن قابلیت اجرای یکسان در محیط های مختلف از طریق

Docker

دامنه سیستم



نقش‌های سیستم

Spectator : تماشاگر کاربر نهایی سامانه است و اجازه انجام عملیات مربوط به حساب خود، رزرو، خرید و درخواست‌های پس از خرید را دارد.

Support : پشتیبان نقش مدیریتی/عملیاتی دارد، اما به جای دسترسی مستقیم و کنترل‌نشده به دیتابیس، عملیات از طریق API های مشخص و Business Rule های کنترل‌شده انجام می‌شود.

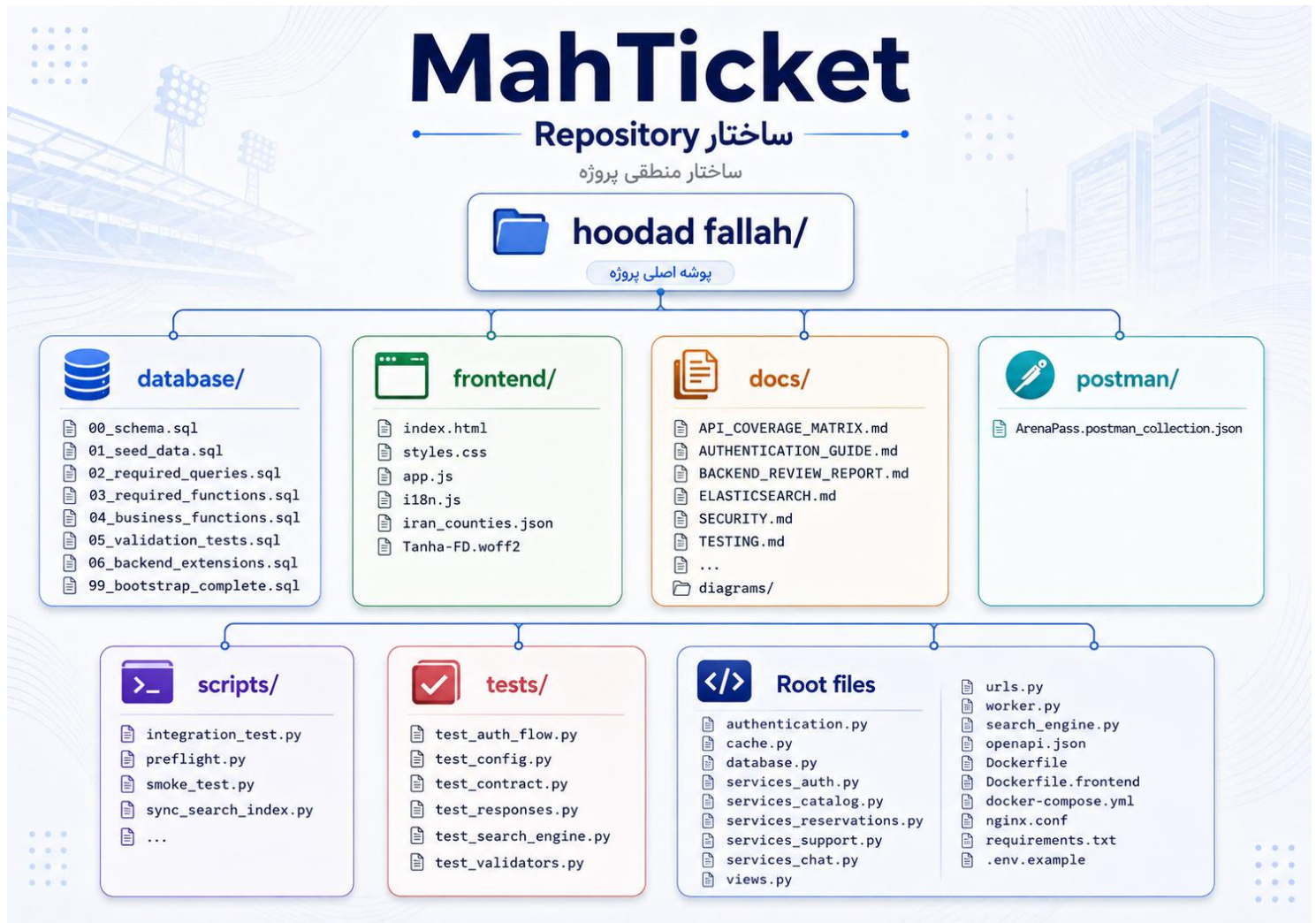
این طراحی مانع آن می‌شود که پشتیبان به صورت دلخواه وضعیت‌های حساس مانند paid را جعل یا منطق مالی سیستم را دور بزند.

فناوری‌های استفاده‌شده :

بخش	فناوری
Backend Runtime	Python 3.13
Web Framework	Django 5.2
Database	PostgreSQL 16
Database Driver	psycopg 3
Connection Pool	psycopg-pool
Cache	Redis 8
Authentication	JWT / PyJWT
Search	Elasticsearch
Application Server	Gunicorn
Frontend	HTML5 / CSS3 / JavaScript
Reverse Proxy	Nginx
Local Mail Testing	Mailpit
Real OTP Email	SMTP / Gmail
Containerization	Docker
Multi-container Runtime	Docker Compose
API Contract	OpenAPI 3.1
API Manual Testing	Postman
Automated Tests	Pytest

ساختار Repository

ساختار منطقی پروژه به شکل زیر است:



طراحی پایگاه داده

MahTicket از PostgreSQL به عنوان دیتابیس اصلی استفاده می‌کند.

دیتابیس فقط محل ذخیره اطلاعات نیست و بخش مهمی از Business Logic حساس سیستم در سطح PostgreSQL محافظت شده است.

در طراحی فعلی حدود ۳۰ جدول عملیاتی و زیرساختی وجود دارد.

جداول جغرافیایی

provinces

نگهداری استان‌ها.

کلید اصلی: id

فیلدهای مهم: name, created_at

cities

هر شهر به یک استان تعلق دارد.

رابطه: Province 1 ——— N City

venues

محل برگزاری مسابقات.

اطلاعات مهم:

- شهر
- نام ورزشگاه/سالن
- آدرس
- ظرفیت
- Latitude
- Longitude
- وضعیت فعال بودن

کاربران و کیف پول

users

جدول کاربران اصلی سیستم است.

فیلدهای مهم:

- first_name .
- last_name .
- email .
- phone .
- password_hash .
- role .
- date_of_birth .
- preferred_login .
- is_active .
- session_version .
- email_verified_at .
- phone_verified_at .
- last_login_at .

Role های معتبر:

spectator

support

حداقل یکی از email یا phone الزامی است.

Email با نوع CITEXT ذخیره می شود تا مقایسه Case-insensitive باشد.

Password به شکل Hash ذخیره می شود و هیچ Password خامی در جدول User ذخیره نمی شود.

wallets

هر کاربر حداکثر یک کیف پول دارد.

رابطه: User 1 ——— 0..1 Wallet

فیلد اصلی: balance

موجودی کیف پول نمی تواند منفی شود.

wallet_transactions

تمام تغییرات موجودی کیف پول به شکل Ledger ثبت می شوند.

این جدول Append-only طراحی شده تا تاریخچه مالی قابل دستکاری نباشد.

ساختار مسابقات

sport_types

نوع ورزش:

- فوتبال
- والیبال
- بسکتبال
- سایر ورزش‌های قابل توسعه

teams

تیم‌ها به Sport Type متصل هستند.

organizers

برگزارکنندگان مسابقه.

matches

هر Match شامل:

- نوع ورزش
- تیم میزبان
- تیم میهمان
- ورزشگاه
- برگزارکننده
- نام لیگ/تورنمنت

- زمان شروع

- زمان پایان

- Status

- وضعیت فعال بودن

سیستم از ایجاد مسابقه با تیم میزبان و میهمان یکسان جلوگیری می‌کند.

مدل بلیط

ticket_categories

رده‌هایی مانند: Normal, VIP, Special, ...

tickets

هر رکورد Ticket یک Bucket یا Seat قابل فروش را نمایش می‌دهد.

فیلدهای مهم:

- match_id

- ticket_category_id

- section_code

- row_code

- seat_code

- is_numbered

- price
- total_capacity
- held_quantity
- sold_quantity
- change_held_quantity
- available_quantity
- sale_starts_at
- sale_ends_at
- is_active

موجودی از چند Counter تشکیل شده است تا Hold های عادی و Hold های مربوط به تغییر جایگاه نیز از هم تفکیک شوند. برای صندلی شماره‌دار: $quantity = 1$ الزامی است.

امکانات بلیط

amenities

امکانات قابل تعریف به صورت مستقل، مانند:

- پارکینگ
- پذیرایی

• جایگاه مسقف

• ورودی اختصاصی

• VIP Service

ticket_amenities

رابط چندبه چند بین Ticket و Amenity است.

Ticket N ——— N Amenity

این طراحی از اضافه شدن ستون های ثابت و غیرقابل توسعه به جدول Ticket جلوگیری می کند.

رزرو

reservations

جدول مرکزی Transaction Workflow است.

Status های اصلی:

Held, paid, canceled, expired, refunded

اطلاعات مهم:

• کاربر

• بلیط

• تعداد

• قیمت واحد

- مبلغ کل
 - زمان رزرو
 - زمان انقضا
 - زمان پرداخت
 - زمان لغو
 - عامل لغو
 - دلیل لغو
 - وضعیت بررسی پشتیبان
- Reservation از زمان ایجاد وارد حالت held می شود.
- Hold به صورت پیش فرض حداکثر ۱۰ دقیقه اعتبار دارد، ولی زمان واقعی Hold از بین سه مقدار زیر کمینه گرفته می شود:
- 10 minutes
- Match start - 1 second
- Ticket sale end
- بنابراین هیچ Hold ی نمی تواند بعد از پایان فروش یا شروع مسابقه معتبر باقی بماند.

پرداخت

payment_methods

روش‌های پرداخت را نگهداری می‌کند.

مانند:

wallet

local_gateway

payments

هر تلاش پرداخت به Reservation مربوط است.

Status پرداخت مستقل از Status رزرو ذخیره می‌شود.

برای هر Reservation حداکثر یک پرداخت Successful مجاز است.

این محدودیت علاوه بر Business Logic با Constraint/Index نیز کنترل می‌شود.

بلیط صادرشده

issued_tickets

پس از پرداخت موفق، بلیط نهایی صادر می‌شود.

هر Issued Ticket شامل:

• ticket_number

• qr_token

• status

• issued_at

• used_at

است.

تعداد Issued Ticket های Reservation پرداخت شده باید دقیقاً با Quantity رزرو برابر باشد.

کنسلی و Refund

cancellation_policies

قانون جریمه برگزارکننده را بر اساس فاصله زمانی تا Match نگهداری می‌کند.

cancellation_requests

درخواست کاربر برای کنسلی.

refunds

نتیجه مالی درخواست کنسلی تأیید شده.

مبلغ Refund طبق رابطه زیر محاسبه می‌شود:

$$\text{Refund} = \text{Payment Amount} - \text{Penalty Amount}$$

مبلغ برگشتی به Wallet کاربر افزوده شده و همزمان Wallet Transaction ثبت می‌شود.

تغییر جایگاه

seat_change_requests

کاربر می‌تواند برای Reservation پرداخت شده درخواست تغییر جایگاه دهد.

برای حفظ سادگی و یکپارچگی مالی، پیاده‌سازی فعلی مقصدهای هم‌قیمت را می‌پذیرد.

شرایط مهم:

- Match مقصد باید همان Match قبلی باشد.
- بلیط مقصد باید فعال باشد.
- Price باید سازگار باشد.
- Inventory مقصد باید موجود باشد.
- مقصد به‌طور موقت Hold می‌شود.
- در صورت رد یا Expire شدن درخواست، Hold مقصد آزاد می‌شود.
- در صورت تأیید، موجودی Old Ticket و New Ticket به‌صورت Transactional انتقال می‌یابد.

گزارش مشکلات

report_categories

موضوع گزارش.

reports

کاربر می‌تواند مشکل مرتبط با:

• Ticket

• Reservation

• Payment

را ثبت کند.

پشتیبان قادر است:

• Report را Assign کند.

• پاسخ ثبت کند.

• وضعیت را تغییر دهد.

• Report را Resolve کند.

تاریخچه وضعیت رزرو

reservation_status_history

هر Transition مهم Reservation ثبت می‌شود.

در نتیجه وضعیت فعلی تنها منبع بررسی Workflow نیست و تاریخچه تغییرات نیز قابل Audit است.

Audit Log

api_audit_log

عملیات حساس API در این جدول ثبت می‌شوند.

اطلاعاتی مانند:

- Actor
- Action
- Resource Type
- Resource ID
- Request ID
- IP
- Metadata
- Timestamp

برای Traceability نگهداری می‌شود.

Support Chat

support_conversations

برای هر تماشاگر یک Conversation اصلی قابل مدیریت ایجاد می‌شود.

اطلاعات:

- Spectator

Support Assignee •

Status •

Subject •

Last Message •

Read State •

support_messages

پیام‌های گفتگو.

ویژگی‌ها:

Sender •

Conversation •

Body •

Created Time •

Read State •

Search Synchronization Outbox

search_sync_outbox

برای جلوگیری از ناسازگاری PostgreSQL و Elasticsearch از Transactional Outbox استفاده شده است.

در نتیجه تغییر Ticket مستقیماً و به شکل غیرقابل اتکا به Elasticsearch وابسته نمی‌شود.

ابتدا تغییر در PostgreSQL Commit می‌شود و Event مربوط به Sync در Outbox قرار می‌گیرد.

Worker سپس Event را پردازش می‌کند.

این طراحی امکان Retry و Recovery را فراهم می‌کند.

Bootstrap State

`system_bootstrap_state`

پس از اجرای موفق تمام SQL های Initialization ، Marker تکمیل Bootstrap در این جدول ثبت می‌شود.

Healthcheck دیتابیس Docker نیز از همین وضعیت استفاده می‌کند تا صرفاً باز بودن Port PostgreSQL را به معنی Ready بودن کامل دیتابیس در نظر نگیرد.

ER Diagram

تصویر در صفحه ی بعد.



روابط اصلی ER

Province 1 — N City

City 1 — N Venue

City 1 — N User

City 1 — N Team

User 1 — 1 Wallet

SportType 1 — N Team

SportType 1 — N Match

Team 1 — N Match

Venue 1 — N Match

Organizer 1 — N Match

Match 1 — N Ticket

TicketCategory 1 — N Ticket

Ticket N — N Amenity

User 1 — N Reservation

Ticket 1 — N Reservation

Reservation 1 — N Payment

Wallet 1 — N WalletTransaction

Reservation 1 — N CancellationRequest

CancellationRequest 1 — 0..1 Refund

Reservation 1 — N SeatChangeRequest

Reservation 1 — N IssuedTicket

User 1 — N Report

Reservation 1 — N ReservationStatusHistory

User 1 — 1 SupportConversation

SupportConversation 1 — N SupportMessage

نرمال سازی

ساختار دیتابیس با هدف رعایت NF ۳ طراحی شده است.

نمونه تصمیمات نرمال سازی:

- Province و City جدا شده اند.
- Venue از Match جداست.
- Sport و Team از Match جدا هستند.
- Payment Method جدول مستقل دارد.
- Ticket Category مستقل است.
- Amenity با جدول واسط چند به چند ذخیره می شود.
- Cancellation Policy از Cancellation Request جدا است.
- Refund از Payment و Request جدا نگهداری می شود.
- Report Category مستقل است.
- Reservation History از Reservation جدا است.

این ساختار باعث کاهش:

• Redundancy

• Update Anomaly

Delete Anomaly •

Insert Anomaly •

می‌شود.

Constraints و یکپارچگی داده

بخش قابل توجهی از صحت سیستم در خود PostgreSQL enforce می‌شود.

نمونه قوانین:

کاربران

• Email یا Phone الزامی است.

• Email یکتا است.

• Phone یکتا است.

• Role فقط spectator یا support.

• Preferred Login باید Contact متناظر داشته باشد.

• Birth Date دارای محدوده منطقی است.

بلیط

• Price منفی نیست.

• Capacity مثبت است.

• Inventory Counter ها ناسازگار نمی‌شوند.

- Seat شماره‌دار Duplicate نمی‌شود.
- Sale End باید بعد از Sale Start باشد.

رزرو

- Expiry بعد از Reservation Time است.
- Quantity مثبت است.
- User باید Spectator فعال باشد.
- بلیط و وابستگی‌های Catalog باید فعال باشند.

پرداخت

- Amount منفی نیست.
- Successful Payment باید با Reservation Amount برابر باشد.
- یک Successful Payment برای هر Reservation .

Refund

- Refund به Payment و Cancellation صحیح متصل است.
- $\text{Refund} + \text{Penalty} = \text{Payment Amount}$.

Wallet

- Balance منفی نمی‌شود.
- Ledger قابل دستکاری نیست.
- Balance نهایی باید با مجموع Ledger سازگار باشد.

Support

- Reviewer های Support Workflow باید واقعاً Role برابر support داشته باشند.

Indexing

برای عملیات پرتکرار Index های متعددی تعریف شده است.
دسته‌های اصلی:

Foreign Key Indexes

برای Join های پرتکرار.

Search Indexes

برای:

• User Name

• Team Name

• Venue

• City

• Tournament

• Category

• Amenity

از pg_trgm نیز استفاده شده است.

Inventory Indexes

برای Ticket های قابل فروش و Reservation های Expiring.

Payment Indexes

برای Status و Payment Time .

Workflow Indexes

برای:

• Cancellation Requests

• Seat Change Requests

• Reports

• Support Queue

• Search Outbox

در Schema اصلی و Backend Extension در مجموع ده ها Index صریح تعریف شده است.

View های دیتابیس

پنج View اصلی وجود دارد:

`v_ticket_catalog`

اطلاعات Ticket را با Match ، Sport ، Teams ، Venue ، City ، Province ، Organizer و Category ترکیب می کند.

v_purchased_tickets

اطلاعات خریدهای موفق را به شکل یک View تحلیلی آماده می‌کند.

football_details

نمای سازگار با جزئیات بلیط فوتبال.

volleyball_details

جزئیات والیبال.

basketball_details

جزئیات بسکتبال.

این View ها پیچیدگی Query های Backend و Query های الزامی پروژه را کاهش می‌دهند.

Query های الزامی فاز دوم

فایل 02_required_queries.sql شامل ۲۲ Query موردنیاز تکلیف است:

Query 1

کاربرانی که هیچ بلیطی رزرو نکرده‌اند.

Technique: NOT EXISTS

Query 2

کاربرانی که حداقل یک خرید موفق دارند.

Technique: JOIN + DISTINCT

Query 3

مجموع پرداخت ماهانه هر کاربر.

Technique: DATE_TRUNC, GROUP BY, SUM

Query 4

کاربرانی که در یک شهر تنها یک خرید داشته‌اند.

Query 5

کاربر صاحب جدیدترین خرید.

Query 6

کاربرانی که مجموع پرداخت آن‌ها بیشتر از میانگین کاربران است.

Technique: CTE, Window AVG

Query 7

تعداد بلیط فروخته شده به تفکیک Sport Type.

Query 8

سه کاربر با بیشترین خرید هفت روز اخیر.

Query 9

فروش بلیط استان تهران به تفکیک شهر.

Query 10

شهرهای خرید قدیمی‌ترین کاربر ثبت‌نام شده.

Query 11

لیست کاربران Support .

Query 12

کاربرانی با حداقل دو بلیط خریداری شده.

Query 13

کاربرانی با یک تا دو بلیط Football .

Query 14

کاربرانی که از هر سه ورزش اصلی خرید داشته‌اند.

Technique: COUNT(DISTINCT sport)

Query 15

خریدهای امروز به ترتیب ساعت.

Query 16

دومین Ticket پرفروش.

Technique: DENSE_RANK

Query 17

پشتیان دارای بیشترین تعداد لغو و درصد سهم لغو.

Query 18

تغییر نام خانوادگی کاربر دارای بیشترین بلیط لغوشده به Reddington .

Query 19

حذف Reservation های canceled کاربر Reddington با در نظر گرفتن وابستگی‌ها.

Query 20

حذف Reservation های canceled سیستم.

این Query در Transaction آزمایشی اجرا شده و Rollback می‌شود تا Seed اصلی تخریب نشود.

Query 21

کاهش ۱۰٪ قیمت Ticket های مربوط به خرید روز قبل در ورزشگاه آزادی.

Query 22

موضوع و تعداد Report برای Ticket دارای بیشترین Report .

Stored Function های الزامی

فایل 03_required_functions.sql هشت Stored Function اصلی:

`sp_get_purchased_tickets_by_contact`

ورودی:

Phone یا Email

خروجی:

خریدهای کاربر به ترتیب زمان.

sp_get_users_canceled_by_support

ورودی:

Contact پشتیبان

خروجی:

کاربرانی که توسط او Reservation لغو شده دارند.

sp_get_purchased_tickets_by_city

ورودی:

نام شهر

خروجی:

Ticket های خریداری شده در آن شهر.

sp_search_tickets

جستجو در:

• نام خریدار

• Team

• Venue

• City

• Sport

• Category

• Section

• Amenity

sp_get_same_city_users

کاربران هم‌شهر یک User.

sp_top_buyers_since

ورودی:

Start Date

N

خروجی:

N خریدار برتر بعد از تاریخ مشخص.

sp_get_canceled_tickets_by_sport

Ticket های لغوشده بر اساس Sport.

sp_top_reporters_by_subject

کاربرانی که در موضوع مشخص بیشترین Report را ثبت کرده‌اند.

Business Function های PostgreSQL

فایل database/04_business_functions.sql شامل :

توابع اصلی:

fn_get_penalty_percentage

release_pending_seat_change_holds

expire_pending_seat_change_requests

expire_pending_reservations

reserve_ticket

process_payment

top_up_wallet

request_cancellation

review_cancellation

request_seat_change

review_seat_change

deactivate_user

مهم‌ترین مزیت قرار دادن این Workflow ها در PostgreSQL این است که عملیات حساس به شکل Atomic اجرا می‌شوند و Client یا Backend نمی‌تواند نیمی از یک تراکنش را انجام دهد.

معماری Backend

Backend با Django پیاده‌سازی شده است اما هیچ Django Model و ORM در پروژه استفاده نشده است.

Django در این پروژه مسئول:

• HTTP Routing

Request Handling •

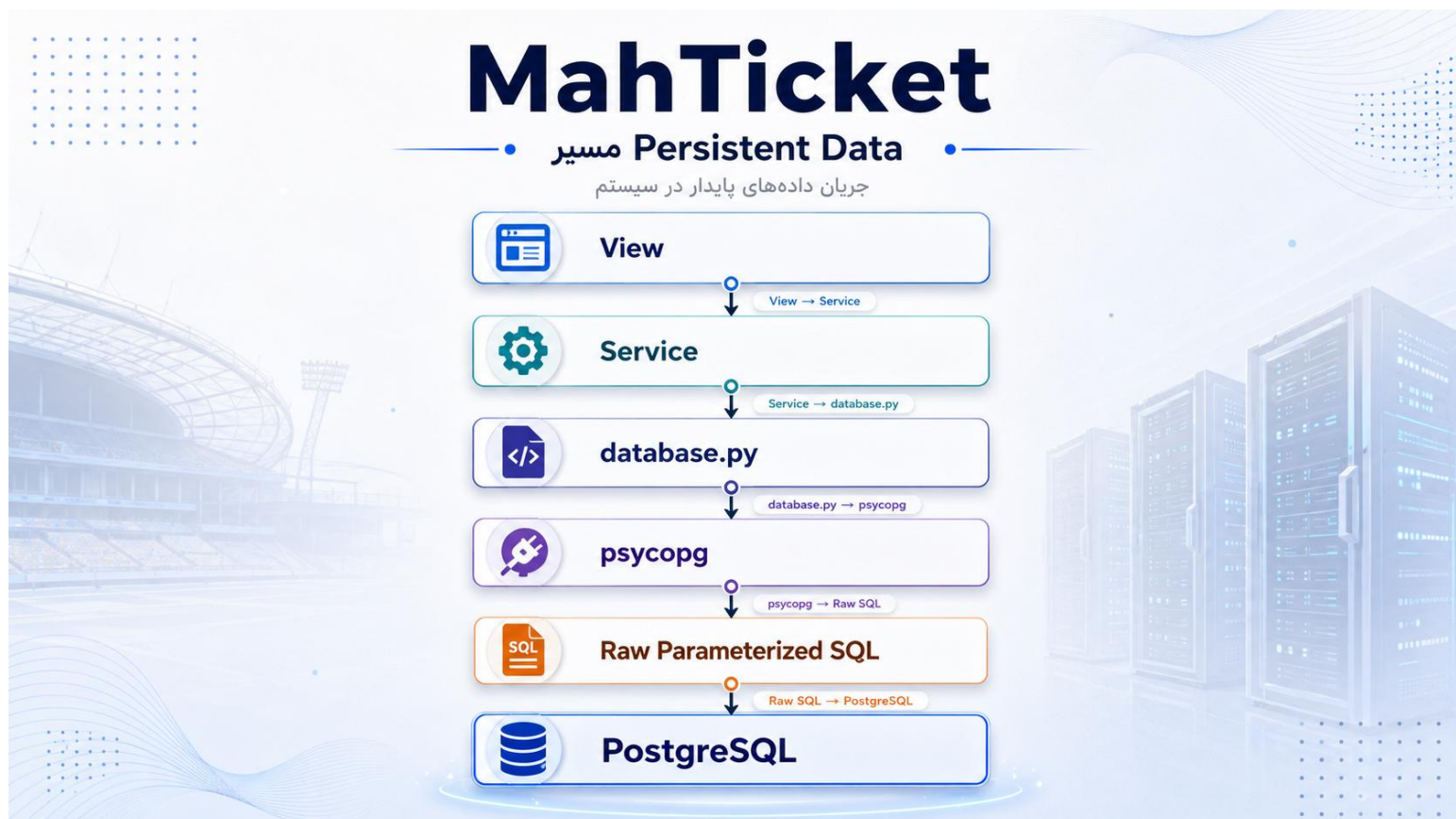
Middleware •

Authentication Wrapper •

JSON Response •

است.

مسیر Persistent Data :



لایه‌های Backend

views.py

مسئول:

- دریافت Request
- بررسی Method
- Parse پارامتر
- Validation
- Permission
- فراخوانی Service
- ساخت Response

services_auth.py

منطق:

- Signup
- Password Login
- OTP Login
- Profile
- Password Change
- Contact Change

services_catalog.py

- Cities .
- Venues .
- Sports .
- Matches .
- Ticket Search .
- Ticket Detail .
- Seat Change Options .

services_reservations.py

- Reservation .
- Payment .
- Wallet .
- Cancellation .
- Seat Change .
- Report .
- Issued Ticket .

services_support.py

تمام عملیات Role پشتیبان.

services_chat.py

گفتگوی تماشاگر/پشتیبان.

database.py

Gateway مرکزی. PostgreSQL

شامل:

• Connection Pool

• Transaction

• fetch_one

• fetch_all

• execute

• execute_many

• ping

cache.py

Gateway مرکزی Redis.

search_engine.py

Elasticsearch Gateway.

worker.py

Background Processing.

API Design

لایه API در سامانه MahTicket بر اساس اصول REST طراحی شده است و تمامی Endpoint های نسخه فعلی سامانه تحت مسیر پایه زیر در دسترس قرار می‌گیرند:

/api/v1

استفاده از Version Prefix در مسیر API این امکان را فراهم می‌کند که در نسخه‌های آینده، تغییرات ناسازگار با نسخه فعلی بدون ایجاد اختلال در Client های موجود مدیریت شوند.

قرارداد رسمی API در فایل زیر تعریف شده است:

openapi.json

این فایل بر اساس استاندارد OpenAPI 3.1.0 تهیه شده و به عنوان مرجع اصلی برای تعریف Endpoint ها، پارامترهای ورودی، مدل‌های داده، ساختار Response ها، وضعیت‌های HTTP و الزامات امنیتی API مورد استفاده قرار می‌گیرد.

در نسخه فعلی MahTicket ، قرارداد OpenAPI شامل:

• 61 API Path

• 66 HTTP Operation

است.

تفاوت تعداد Path ها و Operation ها به این دلیل است که یک Path می‌تواند بیش از یک عملیات HTTP ، مانند GET ، POST ، PUT یا DELETE را ارائه دهد.

این ساختار باعث می‌شود قرارداد بین Frontend و Backend به صورت صریح و قابل بررسی تعریف شده و توسعه، تست و نگهداری API با وابستگی کمتری به پیاده‌سازی داخلی Backend انجام شود.

API Response Contract

برای ایجاد رفتار یکنواخت در تمام Endpoint های سامانه، MahTicket از یک قرارداد استاندارد برای Response های API استفاده می‌کند. به این ترتیب Client می‌تواند بدون وابستگی به نوع Endpoint، پاسخ موفق یا ناموفق را بر اساس ساختاری مشخص پردازش کند.

Successful Response

ساختار عمومی یک پاسخ موفق به شکل زیر است:

```
{  
  
  "success": true,  
  
  "data": {},  
  
  "meta": {}  
}
```

فیلد success نتیجه کلی درخواست را مشخص می‌کند، در حالی که data شامل داده اصلی بازگردانده شده توسط Endpoint است.

فیلد meta برای اطلاعات تکمیلی Response در نظر گرفته شده و بسته به نوع Endpoint می‌تواند شامل اطلاعاتی نظیر Pagination یا سایر Metadata های مورد نیاز Client باشد.

Paginated Response

در Endpoint هایی که خروجی آن‌ها شامل مجموعه بزرگی از داده‌ها است، Pagination مورد استفاده قرار می‌گیرد. ساختار پاسخ این Endpoint ها به صورت زیر است:

```
{  
  "success": true,  
  "data": [],  
  "meta": {  
    "page": 1,  
    "page_size": 20,  
    "total": 100,  
    "pages": 5  
  }  
}
```

در این ساختار:

- page شماره صفحه فعلی را مشخص می‌کند.
- page_size حداکثر تعداد رکوردهای موجود در هر صفحه را نشان می‌دهد.
- total تعداد کل رکوردهای منطبق با Query را مشخص می‌کند.

- pages تعداد کل صفحات قابل دسترس را نمایش می‌دهد.

این قرارداد به Frontend اجازه می‌دهد کنترل‌های Pagination را بدون نیاز به محاسبات یا Query های اضافی ایجاد کند.

Error Response

در صورت ناموفق بودن درخواست، API به جای بازگرداندن ساختارهای متفاوت برای خطاهای مختلف، از یک Error Contract یکپارچه استفاده می‌کند:

```
{  
  
  "success": false,  
  
  "error": {  
  
    "code": "validation_error",  
  
    "message": "Invalid input.",  
  
    "request_id": "..."  
  
  }  
  
}
```

در این ساختار، code یک شناسه قابل پردازش توسط نرم‌افزار برای نوع خطا است و message توضیح قابل نمایش یا قابل استفاده برای Client را ارائه می‌دهد.

فیلد request_id نیز شناسه یکتای درخواست است که امکان دنبال کردن یک Request مشخص را در لایه‌های Logging، Monitoring و Audit فراهم

می‌کند. وجود این شناسه به‌ویژه هنگام بررسی خطاهای Backend و تحلیل رخدادهای عملیاتی اهمیت دارد، زیرا می‌توان یک درخواست را از زمان ورود به سیستم تا پایان پردازش آن ردیابی کرد.

Authentication Architecture

زیرسیستم Authentication در ArenaPass برای پشتیبانی از سناریوهای مختلف ورود و ثبت‌نام طراحی شده است. در نسخه فعلی، سه Workflow اصلی برای احراز هویت وجود دارد:

۱. Password-based Login

۲. OTP-based Login

۳. OTP-Verified Signup

پس از احراز موفق هویت، سامانه یک JWT Token Pair صادر می‌کند که برای مدیریت Session و دسترسی کاربر به Endpoint های محافظت‌شده مورد استفاده قرار می‌گیرد.

Password-Based Login

در روش ورود مبتنی بر Password، کاربر اطلاعات تماس و رمز عبور خود را ارسال می‌کند Backend. پس از دریافت درخواست، ابتدا هویت کاربر و سپس وضعیت Account را بررسی می‌کند.

جریان پردازش به شکل زیر است:

Contact + Password



Verify User Credentials



Check Account State



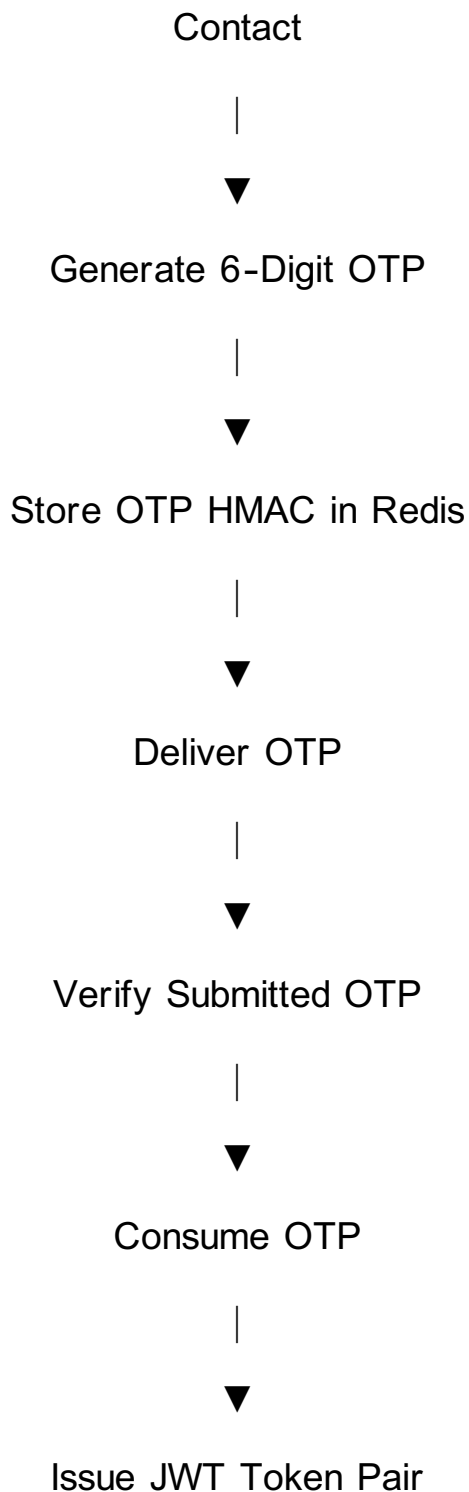
Issue JWT Token Pair

JWT Pair تنها زمانی صادر می‌شود که اطلاعات احراز هویت معتبر بوده و وضعیت حساب کاربری اجازه ورود را بدهد.

OTP-Based Login

MahTicket علاوه بر Password Login ، امکان ورود از طریق One-Time Password (OTP) را نیز فراهم می‌کند.

جریان کلی این فرآیند به شکل زیر است:



OTP تولیدشده به صورت مستقیم در Redis ذخیره نمی شود. در عوض، مقدار HMAC مربوط به آن نگهداری می شود تا در صورت دسترسی غیرمجاز به Storage موقت، مقدار واقعی OTP قابل بازیابی نباشد.

پس از Verification موفق، OTP مصرف شده و امکان استفاده مجدد از آن وجود نخواهد داشت. این رفتار از Replay شدن یک OTP معتبر جلوگیری می کند.

OTP-Verified Signup

فرآیند Signup در ArenaPass به صورت دو مرحله ای طراحی شده است. یکی از تصمیمات مهم معماری در این بخش آن است که پیش از تأیید OTP، هیچ User دائمی در PostgreSQL ایجاد نمی شود.

در مرحله اول، اطلاعات ثبت نام اعتبارسنجی شده، Password هش می شود و State موقت Signup در Redis قرار می گیرد.

Signup Form



Validate Input



Hash Password



Store Temporary Signup State in Redis



Send OTP

پس از دریافت OTP از کاربر، مرحله دوم آغاز می‌شود:

Verify OTP



Consume OTP



Begin SERIALIZABLE Transaction



Create User



Create Wallet



Commit Transaction



Issue JWT Token Pair

ایجاد User و Wallet در یک Transaction با سطح Isolation برابر با SERIALIZABLE انجام می‌شود. بنابراین عملیات ایجاد حساب به صورت اتمیک انجام شده و سیستم از ایجاد وضعیت‌های ناقصی مانند User بدون Wallet جلوگیری می‌کند.

این طراحی همچنین مانع از ایجاد حساب‌های دائمی برای Signup هایی می‌شود که فرآیند تأیید OTP را کامل نکرده‌اند و در نتیجه از تولید داده‌های ناقص یا بلااستفاده در Database جلوگیری می‌شود.

امنیت OTP

- شش‌رقمی است.
- با secrets تولید می‌شود.
- به شکل Plain Text در Redis ذخیره نمی‌شود.
- HMAC آن ذخیره می‌شود.
- TTL دارد.
- تعداد Attempt محدود است.
- Request Rate Limit دارد.
- IP Rate Limit دارد.
- Resend Cooldown دارد.
- پس از Verify موفق Consume می‌شود.

تنظیم پیش فرض

TTL : 300 seconds

JWT

سیستم Access Token و Refresh Token دارد.

Access Token کوتاه عمر است.

Refresh Token طولانی تر است.

هر Token شامل اطلاعاتی مانند:

• User ID

• Role

• Session Version

• JTI

• Issuer

• Audience

• Expiration

است.

Refresh Token ها Rotating هستند.

یعنی هر Refresh موفق:

Old Refresh → Consume

New Access + New Refresh → Issue

اگر Refresh Token قبلی مجدداً استفاده شود، سیستم Replay را تشخیص داده و Token Family را Revoke می‌کند.

Session Version

User دارای session_version است.

در عملیات امنیتی مانند تغییر Password یا Contact ، Session Version تغییر می‌کند.

بنابراین Token های قدیمی حتی در صورت باقی‌بودن زمان Expiration دیگر معتبر نیستند.

Redis

Redis برای چهار گروه اصلی استفاده می‌شود:

OTP

ذخیره State موقت OTP.

Cache

مانند:

• Profile

• City

Venue •

Ticket Search •

Authentication State

Refresh Token •

Revocation •

Token Family •

Blacklist •

Rate Limiting

برای:

OTP •

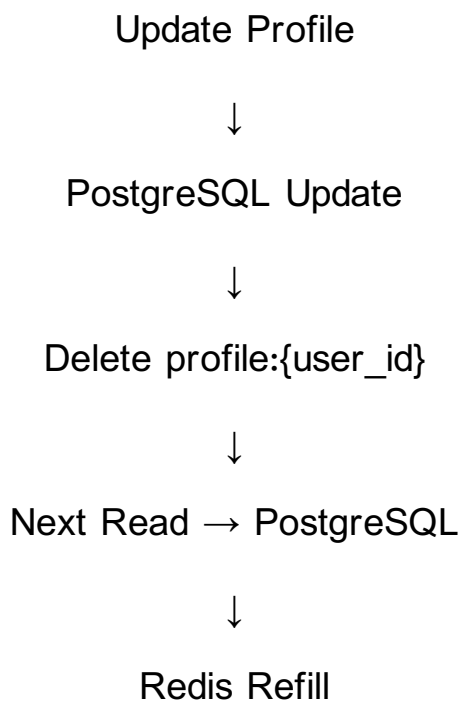
Password Login •

Endpoint های حساس •

Cache Invalidation

یکی از اصول طراحی این است که داده قدیمی Redis بعد از Mutation باقی نماند.

مثلاً:



همین اصل در Catalog و Ticket Search نیز رعایت می‌شود.

جستجوی Ticket

Endpoint: GET /api/v1/tickets

پارامترهای اصلی:

Q, sport, team, city_id, venue_id, category, section, date_from, date_to
price_min, price_max, min_available, numbered, ordering, page,
page_size

Ordering:

starts_at, price, demand, availability

در حالت عادی Search می‌تواند از PostgreSQL استفاده کند.

در حالت فعال بودن Elasticsearch:

Client



API



Elasticsearch

و در صورت خطای Search Engine ، با تنظیم Fallback امکان بازگشت به SQL وجود دارد.

Reservation Workflow

Reservation یکی از حساس‌ترین قسمت‌های سیستم است.

مراحل:

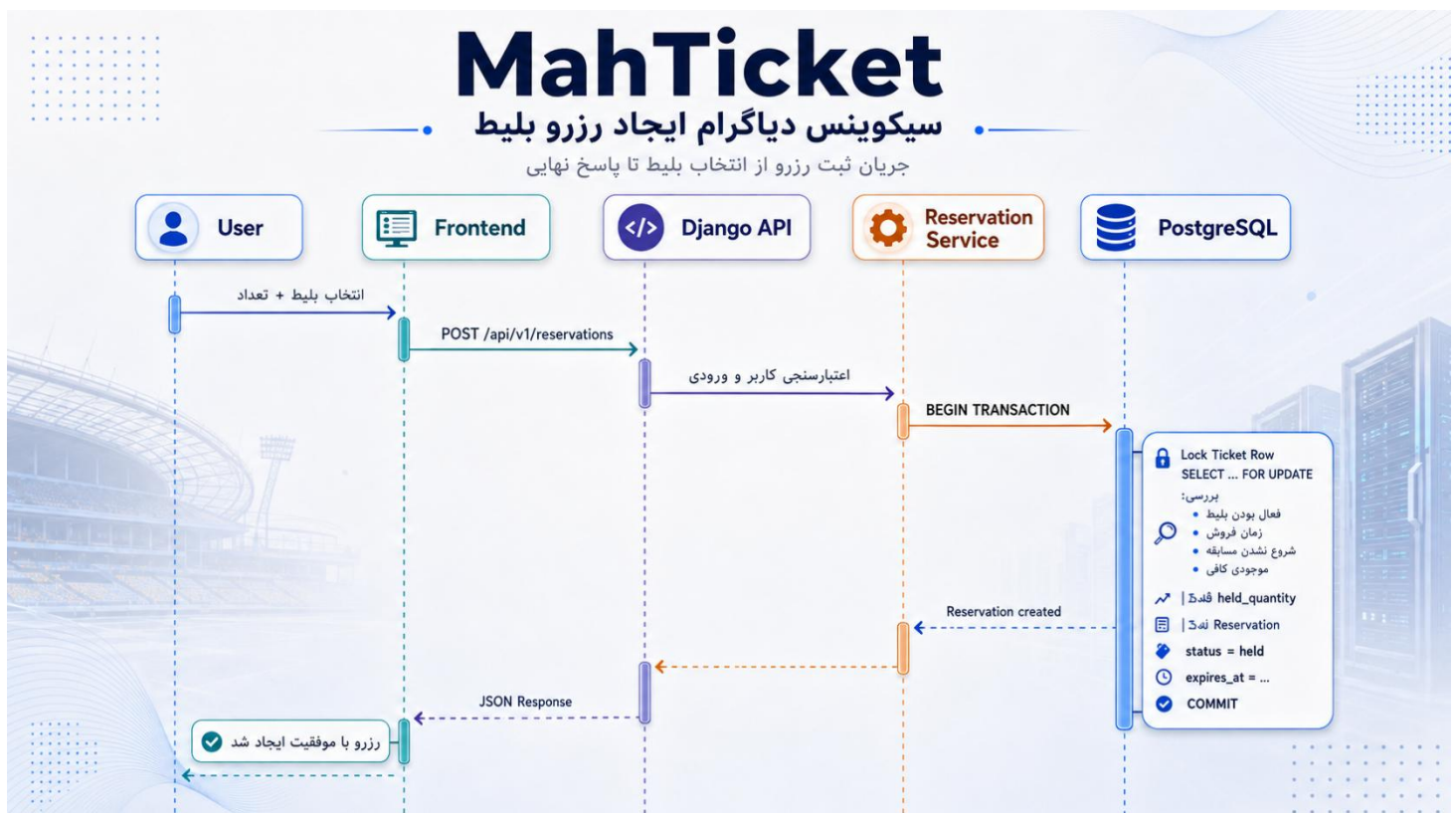
1. Authenticate spectator
2. Validate quantity
3. Lock ticket row
4. Validate ticket/match/catalog state

5. Check sale window
6. Check inventory
7. Increase held_quantity
8. Create Reservation = held
9. Set expiration
10. Commit transaction

استفاده از:

SELECT ... FOR UPDATE

باعث می‌شود دو کاربر همزمان نتوانند یک Inventory محدود را بیش از مقدار واقعی مصرف کنند.



جلوگیری از Overselling

Overselling تنها با بررسی ساده:

$available_quantity > 0$

قابل حل نیست، زیرا دو درخواست همزمان می‌توانند یک مقدار را ببینند.

MahTicket Lock را در Transaction دیتابیس اعمال می‌کند.

بنابراین:

Transaction A → Lock Ticket

Transaction B → Wait

Transaction A → Update inventory → Commit

Transaction B → Read new inventory

Payment Workflow



پرداخت نسبت به Retry حالت Idempotent دارد؛ اگر Payment موفق قبلاً وجود داشته باشد، ایجاد پرداخت موفق دوم مجاز نیست.

Local Gateway

طبق Scope پروژه نیازی به اتصال واقعی بانک نیست.
بنابراین local_gateway یک Payment Simulation محلی است.

این تصمیم باید در دفاع به عنوان Design Constraint مطابق تکلیف توضیح داده شود، نه نقص سیستم.

Wallet

کاربر می‌تواند در محیط Local در صورت فعال بودن تنظیم مربوطه Wallet را شارژ کند.

هر تغییر Wallet همزمان Ledger ایجاد می‌کند.

Cancellation Workflow

ابتدا کاربر Quote می‌گیرد:

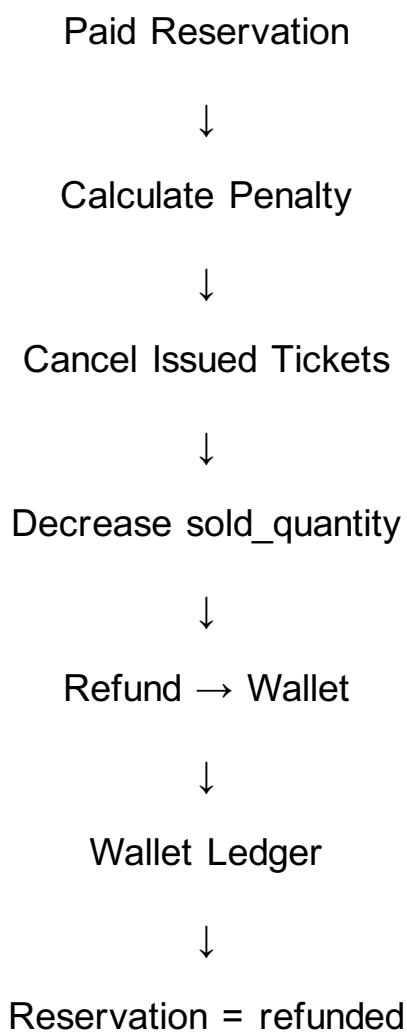
GET cancellation-quote

سپس Request ایجاد می‌کند:

POST cancellation-requests

پشتیبان Request را Review می‌کند.

در حالت Approve :



در Reservation های صرفاً Hold شده، Refund مالی وجود ندارد و Inventory آزاد می‌شود.

Seat Change Workflow

User selects Reservation



Fetch eligible destinations



Select same-match compatible Ticket



Create Seat Change Request



Hold Target



Support Review



Reject Approve



Release Transfer inventory

Hold Update Reservation.ticket_id

Worker درخواست‌های منقضی‌شده را نیز پاکسازی می‌کند.

Support Operations

Support Dashboard اطلاعات عملیاتی سیستم را ارائه می‌دهد.

Support API ها شامل:

- Reservation Review
- Reservation Cancellation
- Seat Correction
- Suspicious Payment Review
- Cancellation Review
- Seat Change Review
- Report Management
- Ticket Management
- User Deactivation
- Chat Management

است.

RBAC در Wrapper های Backend کنترل می‌شود.

Support Chat

گفتگوی پشتیبانی کاملاً در PostgreSQL ذخیره می‌شود.

تماشاگر:

- Conversation را مشاهده می‌کند.
- پیام ارسال می‌کند.
- پاسخ‌ها را Read می‌کند.

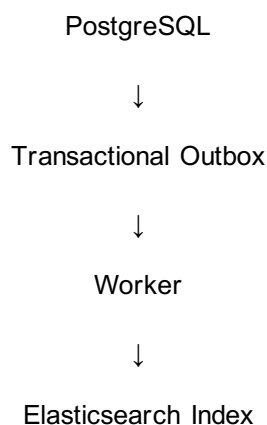
پشتیبان:

- Queue گفتگوها را مشاهده می‌کند.
 - Conversation را باز می‌کند.
 - پاسخ ارسال می‌کند.
 - Conversation را Close/Reopen می‌کند.
- Frontend از Polling سبک HTTP برای بروزرسانی گفتگو استفاده می‌کند.

Elasticsearch

Elasticsearch به عنوان Source of Truth استفاده نشده است.

ساختار:



Search Index دارای Strict Mapping است.

Rebuild کامل Index نیز به شکل:

Create New Index



Bulk Load



Validate



Atomic Alias Swap



Delete Old Index

انجام می شود.

این روش از نمایش Index ناقص در زمان Rebuild جلوگیری می کند.

Search Synchronization

تغییر در موارد زیر می تواند Ticket Search Document را تحت تأثیر قرار دهد:

• Ticket

• Match

• Team

• Sport

• Venue

• City

• Province

• Organizer

• Category

• Amenity

برای این جداول Trigger های Sync تعریف شده‌اند.

Worker Outbox را Batch Processing می‌کند.

در صورت خطا، Event برای Retry باقی می‌ماند.

Worker

Worker یک Process مستقل از HTTP Backend است.

وظایف اصلی:

Reservation Expiration

Reservation های Hold شده که Expire شده‌اند لغو شده و Inventory آزاد می‌شود.

Seat Change Expiration

Target Hold آزاد می‌شود.

Search Outbox

Event های Elasticsearch پردازش می‌شوند.

Payment Reminder

قابلیت Reminder اختیاری است و از طریق Configuration فعال می‌شود.

Frontend

Frontend بدون Framework خارجی ساخته شده است.

فایل‌ها:

frontend/index.html

frontend/styles.css

frontend/app.js

frontend/i18n.js

مزایا:

- عدم نیاز به npm
- عدم نیاز به Build Pipeline
- اجرای ساده
- وابستگی Runtime بسیار کم

صفحات و قابلیت‌های Frontend

Home

- معرفی سامانه
- Search سریع
- Sport Categories
- Featured Matches

Ticket Search

- Filter Panel
- Search
- Sorting
- Pagination
- Active Filters
- Skeleton Loading
- Empty State
- Error State

Ticket Detail

- Match
- Venue
- Price



Seat .

Category .

Amenities .

Quantity .

Total Price .

Authentication Dialog

Password Login .

OTP Login .

Signup .

OTP Verification .

Spectator Dashboard

Account Overview .

Wallet .

Reservations .

Payments .

Bookings .

Issued Tickets .

Cancellation .

Seat Change .

Reports •

Profile •

Support Dashboard

Operational Dashboard •

Reservations •

Cancellation Requests •

Seat Changes •

Reports •

Payments •

Tickets •

Chats •

Responsive Design

Frontend برای Desktop, Tablet, Mobile طراحی شده است.

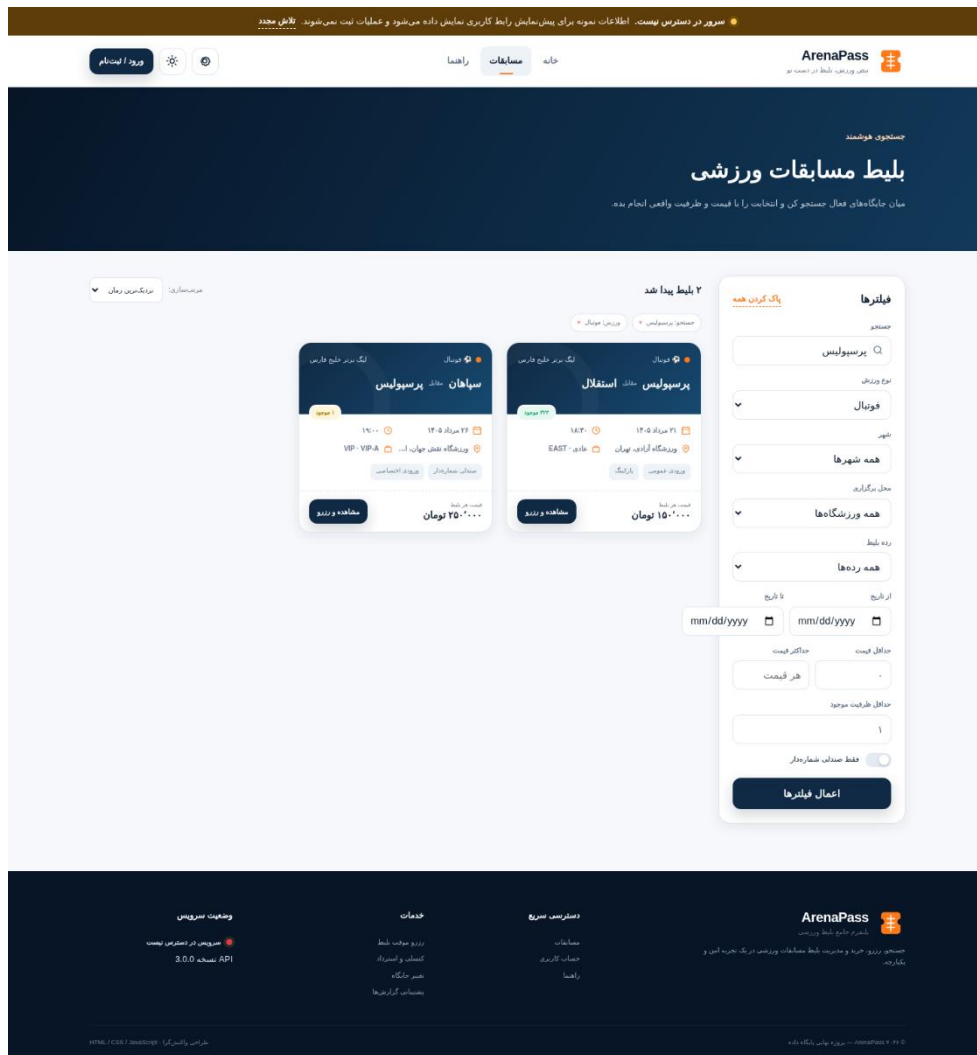
در Mobile:

• Menu تغییر ساختار می‌دهد.

• Filter به Drawer تبدیل می‌شود.

• Dialog ها Responsive می‌شوند.

• Chat Panel فضای بیشتری از Viewport می‌گیرد.



نسخه دسکتاپ :



نسخه موبایل :

زبان و RTL/LTR

Frontend از فارسی و انگلیسی پشتیبانی می‌کند.

کلید:

FA / EN

انتخاب کاربر ذخیره می‌شود.

هنگام تغییر زبان:

- متن‌ها Translate می‌شوند.
- dir تغییر می‌کند.
- Layout بین RTL/LTR جابه‌جا می‌شود.
- Dynamic Content نیز از Translation Layer استفاده می‌کند.

Dark/Light Theme

Theme انتخابی کاربر در Browser Storage نگهداری می‌شود.

Frontend از هر دو Theme روشن و تیره پشتیبانی می‌کند.